

INFORME FINAL

Predicción de cancelaciones de reservas hoteleras

Sistema automático de clasificación binaria, comparación de modelos e inferencia

RESULTADO PRINCIPAL
Random Forest optimizado • ROC-AUC 0,9609 • F1 0,8534 • Accuracy 89,53 %

Equipo	Responsabilidad principal
Daniel Aguilera	Arquitectura MLOps, pipeline, XGBoost y documentación
Nitin Babani	Modelos lineales y árboles, tuning e interpretabilidad
Luis Torres	Deep Learning, Random Forest y alta cardinalidad

Máster en Inteligencia Artificial, Cloud Computing y DevOps
Machine Learning y Deep Learning • Tutor: Sergio Benito Martín

Repositorio: <https://github.com/TFM-JUPITER-ASISTENTE-INDUSTRIAL/practica-machine-learning>

Resumen ejecutivo

El proyecto resuelve un problema real de clasificación binaria: anticipar si una reserva hotelera será cancelada. Sobre 119.390 registros históricos se construyó un flujo modular que limpia los datos, evita fuga de información, transforma 29 variables predictoras en 104 características, entrena cinco familias de modelos y selecciona automáticamente la mejor según ROC-AUC.

DECISIÓN DE MODELO
Random Forest optimizado fue seleccionado por alcanzar el mayor ROC-AUC (0,9609) y el mejor F1 (0,8534). En 23.878 reservas de test identifica 7.276 cancelaciones reales, con 931 falsas alarmas y 1.569 cancelaciones no detectadas.

La solución supera el ejercicio académico y llega hasta inferencia: serializa el preprocesador y el modelo ganador, ofrece predicción individual y por lotes, traduce probabilidades a acciones de Revenue Management y mantiene trazabilidad de experimentos.

Dimensión	Resultado
Datos	119.390 reservas · objetivo 37,04 % canceladas
Validación	Split estratificado 80/20 · test independiente de 23.878 casos
Comparación	Regresión logística, árbol, Random Forest, XGBoost y MLP Keras
Métrica principal	ROC-AUC; F1 como métrica secundaria
Ganador	Random Forest (Tuned): ROC-AUC 0,9609
Extras	MLflow Tracking + interfaz Streamlit

Alineación con el enunciado

- Implementación y comparación coherente de los cinco modelos obligatorios.
- Matriz de confusión, curvas ROC, comparación global e importancia de variables.
- Pipeline reproducible desde el CSV hasta la inferencia y selección automática.
- README, requirements, estructura modular, artefactos persistidos y repositorio Git.
- Dos extras funcionales: registro de experimentos con MLflow e interfaz visual Streamlit.

Índice

Contenido

Resumen ejecutivo.....	2
Alineación con el enunciado.....	2
Índice.....	3
1. Equipo, problema y valor de negocio.....	5
1.1 Roles y contribuciones.....	5
1.2 Justificación del problema.....	5
1.3 Variable objetivo y decisión.....	5
2. Análisis exploratorio de datos.....	6
2.1 Dimensiones, objetivo y calidad.....	6
2.2 Limpieza y reducción de cardinalidad.....	6
2.3 Señales de intención del cliente.....	6
2.4 Depósito y canal comercial.....	7
3. Diseño del sistema y metodología.....	8
3.1 Separación de entrenamiento y test.....	8
3.2 Preprocesamiento.....	8
3.3 Automatización.....	8
4. Modelos y optimización.....	10
4.1 Cinco enfoques complementarios.....	10
4.2 Conexión con las clases.....	10
4.3 Métricas elegidas.....	10
5. Resultados y comparación.....	11
5.1 Resultados en el conjunto de test.....	11
5.2 Lectura comparativa.....	11
6. Reports de evaluación de los modelos.....	12
6.1 Curvas ROC comparativas.....	12
6.2 Evaluación del modelo ganador.....	12
6.3 Importancia de variables.....	13
7. Selección final e inferencia de negocio.....	14
7.1 Elección final.....	14
7.2 Matriz de decisión operativa.....	14
7.3 Prueba real de inferencia.....	14
8. Extra 1 — Registro de experimentos con MLflow.....	15
Valor añadido.....	15
9. Extra 2 — Aplicación visual con Streamlit.....	16
9.1 Funcionalidades.....	16
9.2 Decisiones de ingeniería.....	16
9.3 Flujo de uso.....	16
10. Conclusiones vinculadas a las clases.....	17

10.1 Aprendizajes técnicos.....	17
10.2 Aprendizajes sobre evaluación.....	17
10.3 Conclusión de negocio.....	17
11. Reflexión crítica, limitaciones y mejoras.....	18
11.1 Limitaciones actuales.....	18
11.2 Mejoras priorizadas.....	18
12. Reproducibilidad y entrega.....	19
12.1 Ejecución.....	19
12.2 Artefactos generados.....	19
12.3 Repositorio.....	19
Anexo A – Trazabilidad con la rúbrica.....	20
Fuentes internas consultadas.....	20

1. Equipo, problema y valor de negocio

1.1 Roles y contribuciones

Integrante	Rol	Contribuciones
Daniel Aguilera	Arquitectura MLOps y pipeline	Carga, preprocesado, orquestador, evaluación, XGBoost y documentación.
Nitin Babani	Modelado Machine Learning	Regresión logística, árbol de decisión, tuning, análisis de métricas e interpretabilidad.
Luis Torres	Deep Learning y ensambles	MLP Keras, Dropout, EarlyStopping, Random Forest y tratamiento de agent.

1.2 Justificación del problema

Una cancelación tardía o un no-show convierte inventario perecedero en una habitación vacía. La predicción anticipada permite ajustar el overbooking, reconfirmar reservas, solicitar garantías y priorizar acciones comerciales. El objetivo no es únicamente acertar una etiqueta, sino ordenar reservas por riesgo para que el hotel actúe con umbrales distintos según temporada, ocupación y coste del error.

1.3 Variable objetivo y decisión

La variable `is_canceled` vale 1 cuando la reserva se cancela y 0 cuando se mantiene. Las métricas de test usan un umbral de 0,50 para convertir probabilidad en clase. La aplicación conserva además la probabilidad y la transforma en riesgo bajo (<40 %), medio (40–69 %) o alto (≥ 70 %), lo que permite desacoplar calidad predictiva y política operativa.

PREVENCIÓN DE DATA LEAKAGE

`reservation_status` y `reservation_status_date` se eliminan antes del entrenamiento porque solo se conocen después del desenlace. Mantenerlas produciría métricas artificialmente altas e inútiles en producción.

2. Análisis exploratorio de datos

2.1 Dimensiones, objetivo y calidad

El dataset original contiene 119.390 reservas y 32 columnas. La clase positiva representa 44.224 cancelaciones (37,04 %) y la negativa 75.166 reservas mantenidas (62,96 %). El desbalance es moderado: accuracy por sí sola no basta, pero tampoco obliga a sobremuestrear sin antes comparar métricas de negocio.

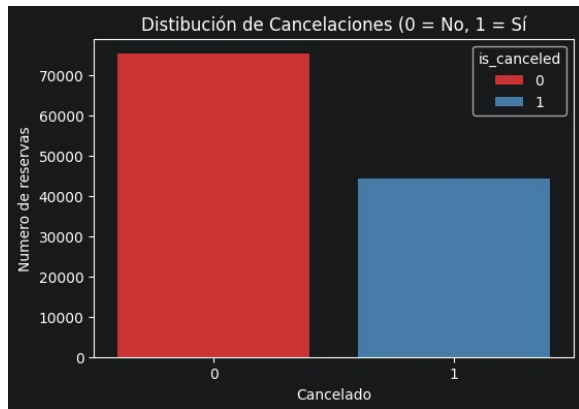


Figura 1. Distribución de la variable objetivo.

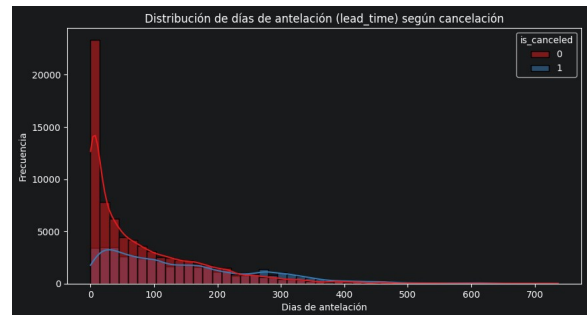


Figura 2. Antelación de reserva según cancelación.

El lead_time separa de forma visible ambos comportamientos: a medida que crece la antelación aumenta la incertidumbre y, por encima de unos 250 días, las cancelaciones llegan a dominar. Esta señal coincide con que lead_time sea la variable más importante del modelo ganador.

2.2 Limpieza y reducción de cardinalidad

Hallazgo	Tratamiento aplicado	Razón
children: 4 nulos	Imputación a 0	Ausencia interpretada como ningún niño.
agent: 16.340 nulos	0 y Top agentes; resto Other	Controlar cardinalidad y distinguir canal directo.
company: 112.593 nulos	Indicador has_company	Conservar información sin miles de identificadores.
country: 177 categorías	Top 5 + Other	El Top 5 concentra 72,9 % de reservas.
Undefined categórico	Sustitución por categoría coherente	Evitar una categoría semánticamente vacía.
ADR -6,38 y 5.400	Filtrado de dos registros extremos	Evitar distorsión por errores detectados en EDA.

2.3 Señales de intención del cliente

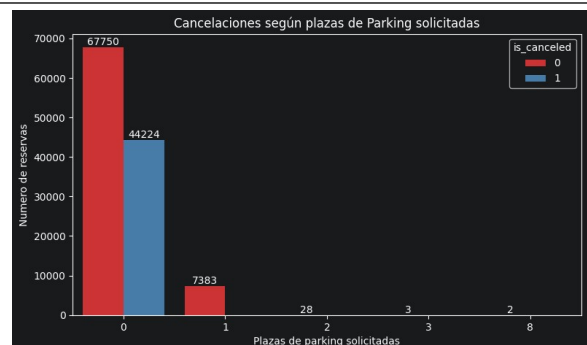
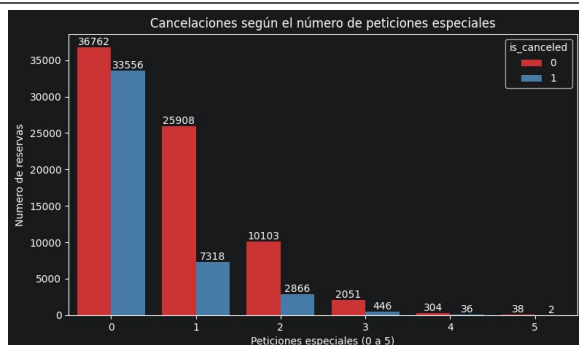


Figura 3. Peticiones especiales y cancelación.

Figura 4. Parking solicitado y cancelación.

Las peticiones especiales y el parking funcionan como señales de compromiso: cuantos más preparativos realiza el cliente, menor es la cancelación observada. La lectura es predictiva, no causal; estas variables ayudan a estimar riesgo, pero no prueban que solicitar un servicio evite cancelar.

2.4 Depósito y canal comercial

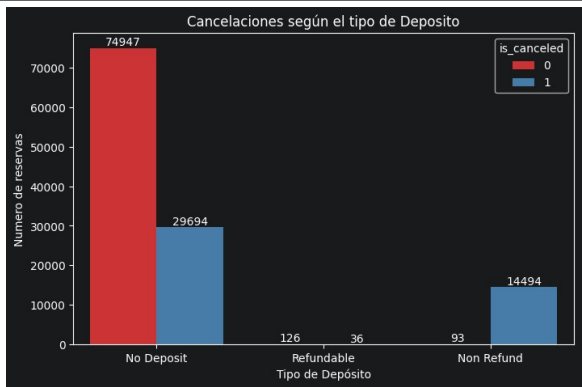


Figura 5. Tipo de depósito.

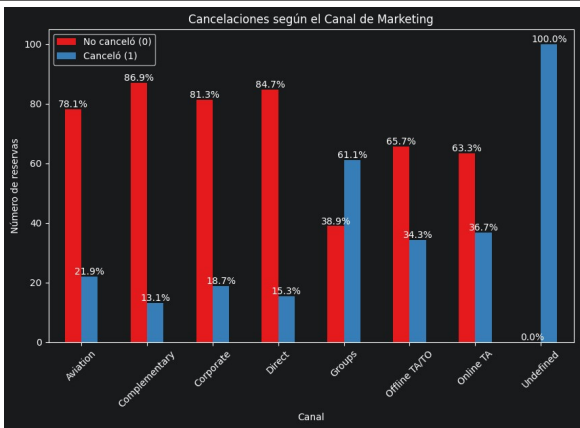


Figura 6. Segmento de mercado.

El patrón Non Refund es contraintuitivo: presenta una tasa de cancelación muy elevada y acaba entre las señales más influyentes. Puede reflejar reservas masivas, intermediarios o procesos administrativos concretos. Los canales directo y corporativo muestran mayor fidelidad, mientras que Groups y algunos canales de agencia concentran más riesgo. Estas relaciones justifican modelos capaces de capturar interacciones no lineales.

3. Diseño del sistema y metodología

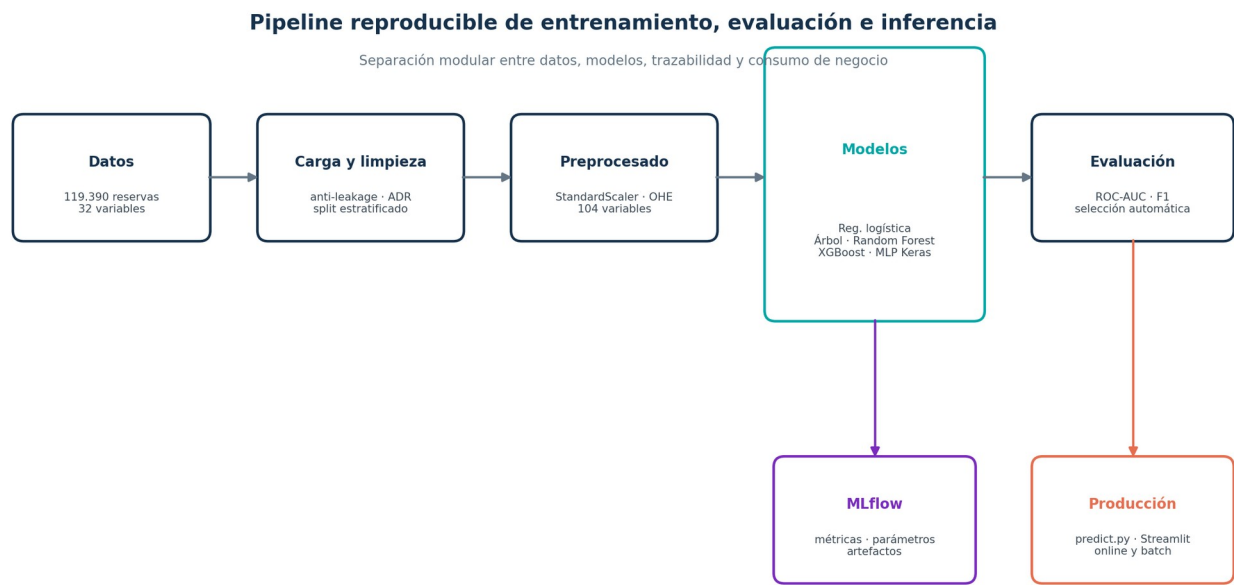


Figura 7. Arquitectura modular del flujo completo.

3.1 Separación de entrenamiento y test

Tras filtrar dos valores anómalos de ADR quedan 119.388 observaciones. El split estratificado y reproducible (random_state=42) reserva 95.510 filas para entrenamiento y 23.878 para test. La estratificación conserva la proporción de cancelaciones: 35.378 positivas en train y 8.845 en test. El conjunto de test se utiliza una sola vez para la comparación final.

3.2 Preprocesamiento

1. clean_features aplica reglas de calidad definidas en el EDA.
2. StandardScaler estandariza las variables numéricas, especialmente útil para regresión logística y red neuronal.
3. OneHotEncoder(handle_unknown="ignore") transforma las categóricas y permite categorías no vistas en inferencia.
4. El ColumnTransformer se ajusta solo con train y transforma después train y test, evitando transferencia de información.
5. El resultado contiene 104 características y el preprocesador se guarda en models/preprocessor.joblib.

3.3 Automatización

src/train.py actúa como orquestador: crea directorios, carga datos, ajusta el preprocesador, construye o optimiza modelos, entrena, calcula métricas comunes, selecciona el máximo ROC-AUC, genera gráficos y registra la ejecución. Esta estructura responde al enfoque de software solicitado en el enunciado y evita concentrar el sistema en un notebook.

Módulo	Responsabilidad
data_loader.py	Lectura, filtrado, exclusión de leakage y split 80/20.

preprocessor.py	Limpieza reutilizable, escalado y codificación categórica.
models/*.py	Construcción y optimización desacoplada de cinco algoritmos.
evaluate.py	Métricas, matriz de confusión, ROC e importancia.
train.py	Orquestación, comparación, selección y persistencia.
predict.py	Carga de artefactos e inferencia online o batch.
app.py	Interfaz de negocio y visualización técnica.

4. Modelos y optimización

4.1 Cinco enfoques complementarios

Modelo	Qué aporta	Optimización
Regresión logística	Baseline lineal, rápido e interpretable.	GridSearchCV: C, iteraciones, class_weight.
Árbol de decisión	Reglas no lineales fáciles de explicar.	GridSearchCV: profundidad, criterio, splits y hojas.
Random Forest	Bagging de árboles; reduce varianza y mejora robustez.	RandomizedSearchCV: árboles, profundidad, muestras, features y pesos.
XGBoost	Boosting secuencial que corrige errores previos.	RandomizedSearchCV: learning rate, profundidad, subsampling y peso positivo.
MLP Keras	Capas densas ReLU y salida sigmoide para relaciones complejas.	Tres arquitecturas; Dropout, Adam y EarlyStopping por val_auc.

4.2 Conexión con las clases

La implementación traslada directamente los principios trabajados en los Jupyter del módulo: modelos supervisados para clasificación binaria, división train/test, estandarización, codificación de categóricas, métricas más allá de accuracy, búsqueda de hiperparámetros y regularización de redes neuronales.

- Random Forest aplica bagging: muchos árboles reducen la varianza del árbol individual y votan una predicción más estable.
- XGBoost aplica boosting: cada ronda corrige errores acumulados, con learning_rate y profundidad como controles principales.
- La red neuronal usa activación sigmoide y binary_crossentropy, adecuadas para una probabilidad binaria.
- Dropout y EarlyStopping limitan el sobreajuste y detienen el entrenamiento cuando deja de mejorar la validación.
- La optimización se realiza en validación cruzada o validación interna; el test queda reservado para estimar generalización.

4.3 Métricas elegidas

Métrica	Lectura en este caso
ROC-AUC (principal)	Capacidad de ordenar cancelaciones por riesgo en todos los umbrales. Permite variar la política operativa.
F1 (secundaria)	Media armónica de precision y recall; penaliza desequilibrios entre falsas alarmas y cancelaciones perdidas.
Precision	De las reservas marcadas como cancelación, proporción realmente cancelada.
Recall	De todas las cancelaciones reales, proporción detectada.
Accuracy	Acierto global; útil como contexto, insuficiente con clases desiguales.

5. Resultados y comparación

5.1 Resultados en el conjunto de test

Modelo	ROC-AUC	F1	Accuracy	Precision	Recall
Random Forest (Tuned)	0,9609	0,8534	89,53 %	88,66 %	82,26 %
XGBoost (Tuned)	0,9582	0,8500	88,42 %	81,67 %	88,62 %
Red Neuronal (Tuned)	0,9537	0,8331	87,80 %	84,45 %	82,20 %
Decision Tree (Tuned)	0,9289	0,7883	85,16 %	83,57 %	74,60 %
Logistic Regression (Tuned)	0,9061	0,7730	82,16 %	73,10 %	82,02 %

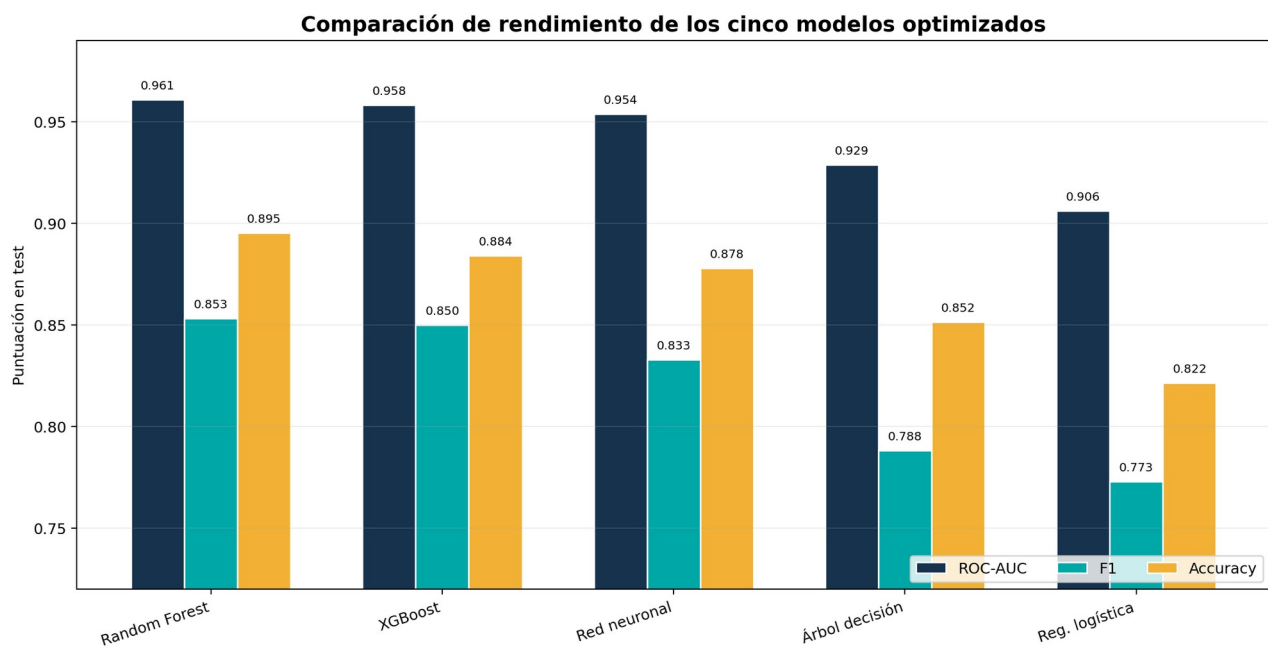


Figura 8. Comparación de ROC-AUC, F1 y accuracy de los modelos optimizados.

5.2 Lectura comparativa

- Random Forest lidera ROC-AUC, F1, accuracy y precision: ofrece el equilibrio global más sólido.
- XGBoost queda a solo 0,0027 de AUC y obtiene el mayor recall (88,62 %); sería preferible si el coste de no detectar una cancelación fuese dominante.
- La red neuronal logra un rendimiento alto, pero no supera a los ensambles de árboles en estos datos tabulares.
- El árbol individual pierde recall por su mayor varianza y menor capacidad de agregación.
- La regresión logística confirma que existen relaciones e interacciones no lineales que un límite lineal no captura por completo.

6. Reports de evaluación de los modelos

6.1 Curvas ROC comparativas

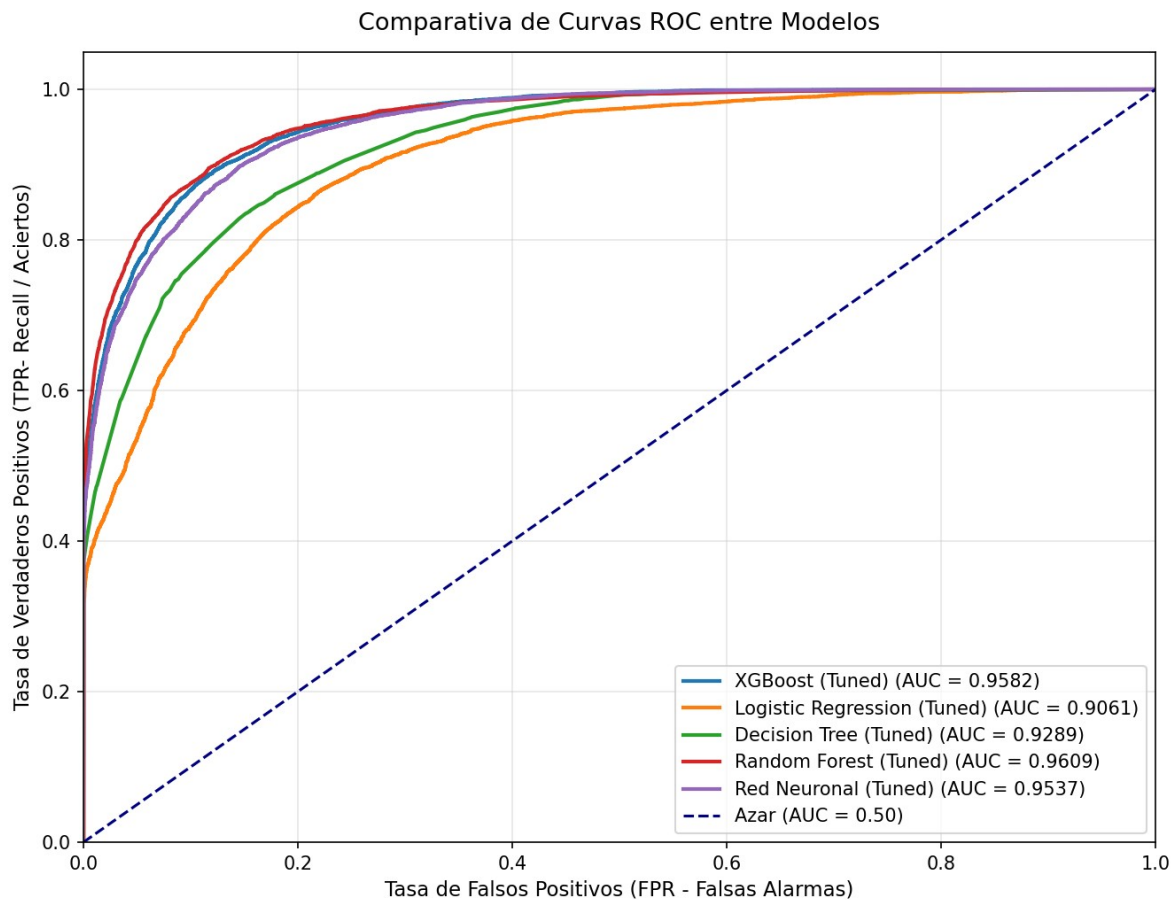


Figura 9. Curva ROC conjunta de los cinco modelos; la diagonal representa clasificación aleatoria.

Todas las curvas se sitúan claramente por encima del azar. Random Forest mantiene la mayor área, aunque XGBoost y la red neuronal se encuentran muy próximos. La curva permite elegir un punto operativo diferente de 0,50 sin volver a entrenar el modelo.

6.2 Evaluación del modelo ganador

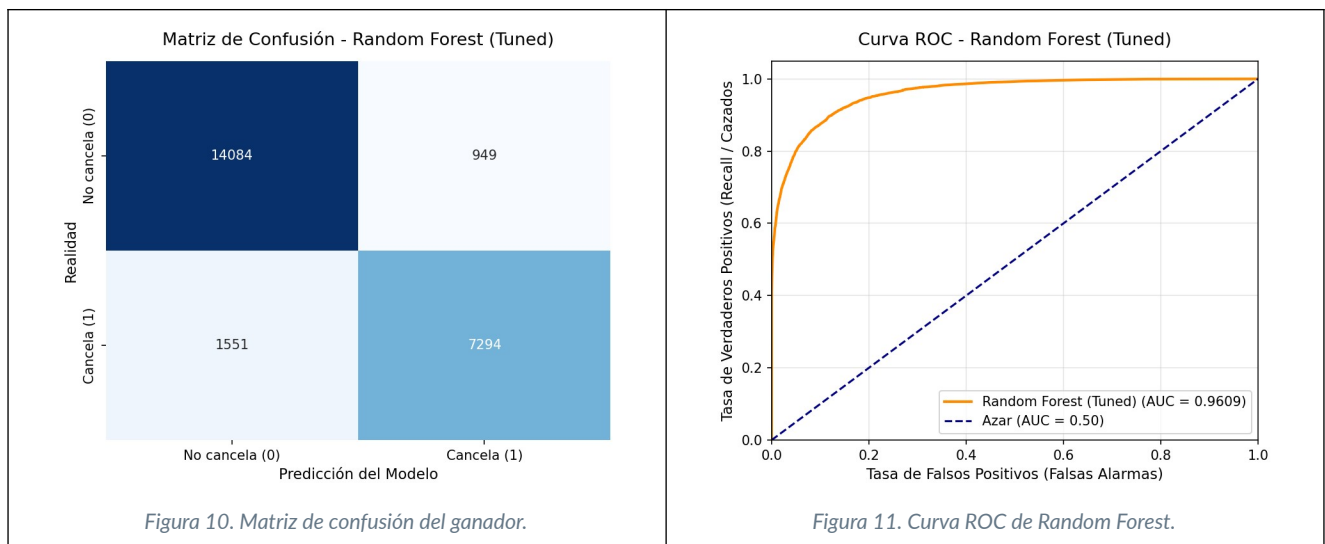


Figura 10. Matriz de confusión del ganador.

Figura 11. Curva ROC de Random Forest.

Resultado	Cantidad	Interpretación
Verdaderos negativos	14.102	Reservas mantenidas clasificadas correctamente.
Falsos positivos	931	Alertas de cancelación que finalmente se mantienen.
Falsos negativos	1.569	Cancelaciones no detectadas por el umbral 0,50.
Verdaderos positivos	7.276	Cancelaciones identificadas correctamente.

La precisión de 88,66 % limita falsas alarmas y el recall de 82,26 % captura más de ocho de cada diez cancelaciones. El equilibrio queda resumido por $F1=0,8534$. Para temporada alta puede bajarse el umbral y ganar recall; para evitar sobreventa puede elevarse y priorizar precisión.

6.3 Importancia de variables

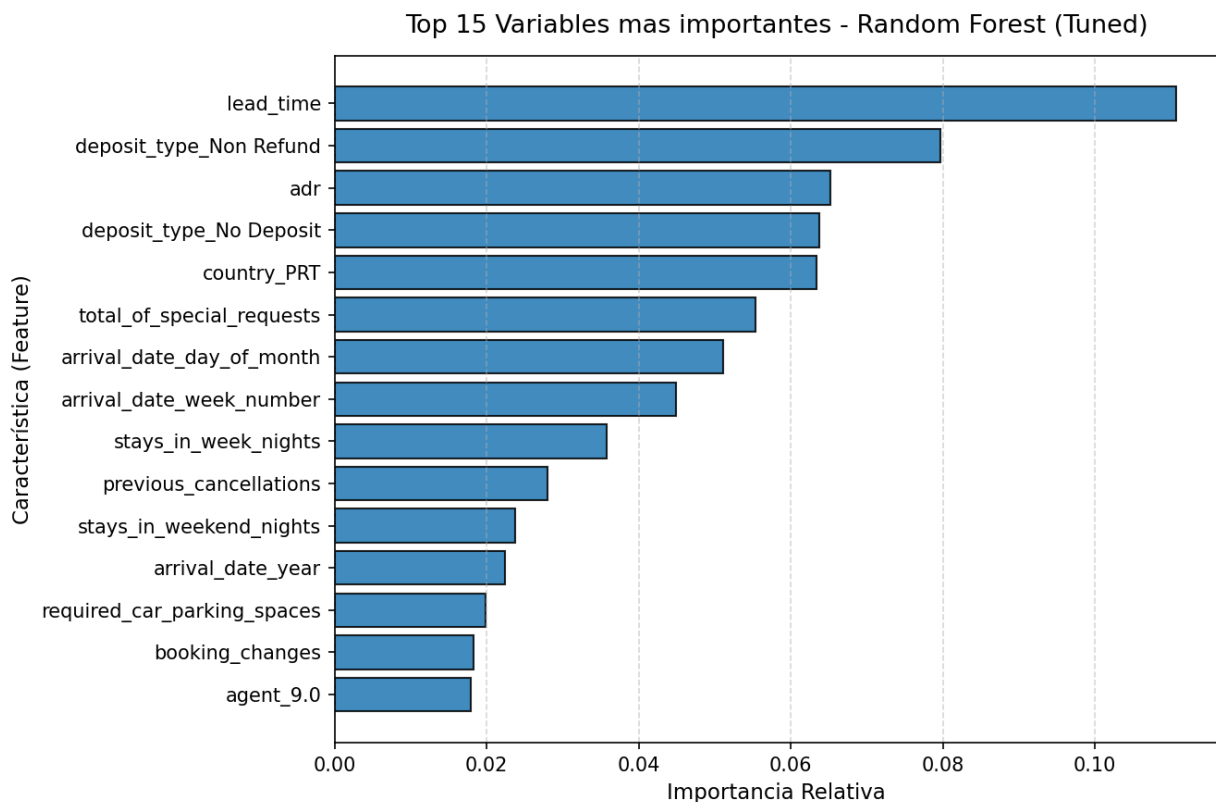


Figura 12. Top 15 de importancias del Random Forest optimizado.

Lead time encabeza la importancia (0,1107), seguido de Non Refund (0,0797), ADR (0,0652), No Deposit (0,0638), país PRT (0,0634) y peticiones especiales (0,0553). El gráfico mide cuánto usa el bosque una variable para reducir impureza; no indica dirección ni causalidad. La dirección se contrasta con el EDA y, como mejora, debería verificarse con permutation importance o SHAP.

7. Selección final e inferencia de negocio

7.1 Elección final

MODELO DE PRODUCCIÓN

Random Forest (Tuned) se serializa en `models/best_model.joblib` junto con el preprocesador. La selección se realiza automáticamente por mayor ROC-AUC, sin fijar a mano el nombre del algoritmo.

La elección prioriza discriminación y equilibrio. XGBoost ofrece mayor recall, pero el Random Forest reduce las falsas alarmas con 6,99 puntos adicionales de precision y mantiene un recall competitivo. En un hotel, esta combinación evita tanto dejar habitaciones vacías como sobre-vender reservas que sí iban a llegar.

7.2 Matriz de decisión operativa

Riesgo	Probabilidad	Acción recomendada
Bajo	< 40 %	Mantener la reserva asignada y evitar fricción innecesaria.
Medio	40–69 %	Enviar reconfirmación de cortesía 48 horas antes.
Alto	≥ 70 %	Valorar overbooking preventivo y solicitar garantía o prepagó.

7.3 Prueba real de inferencia

Caso	Perfil	Probabilidad	Salida
Seguro	Directo, fidelizado y con parking	2,5 %	BAJO · mantener habitación.
Alto riesgo	Agencia, Non Refund y alta antelación	89,0 %	ALTO · garantía y protección de ingresos.

Estas predicciones se obtuvieron ejecutando `src.predict` contra los artefactos persistidos, lo que verifica que preprocesador y modelo pueden reutilizarse fuera del proceso de entrenamiento.

8. Extra 1 — Registro de experimentos con MLflow

EVIDENCIA FUNCIONAL

El experimento `Hotel_Cancellations_PontIA` contiene seis runs finalizadas: una por modelo optimizado y una corrida resumen del ganador.

MLflow aporta trazabilidad y reproducibilidad: cada entrenamiento registra métricas, parámetros y estado; la corrida global añade el CSV comparativo, la curva ROC y los artefactos de producción. Esto permite auditar por qué se eligió un modelo, comparar cambios de hiperparámetros y recuperar resultados sin depender de la salida de terminal.

Run	ROC-AUC	F1	Estado
Random Forest (Tuned)	0,9609	0,8534	FINISHED
XGBoost (Tuned)	0,9582	0,8500	FINISHED
Red Neuronal (Tuned)	0,9537	0,8331	FINISHED
Decision Tree (Tuned)	0,9289	0,7883	FINISHED
Logistic Regression (Tuned)	0,9061	0,7730	FINISHED
Modelo ganador y comparativa	0,9609	—	FINISHED

Valor añadido

- Comparación ordenable de runs y métricas desde una interfaz web.
- Registro de hiperparámetros para reconstruir el experimento.
- Persistencia centralizada de métricas, gráfica ROC, tabla y modelo final.
- Base preparada para control de versiones, model registry y despliegue continuo.

Ejecución: `mlflow ui` · panel local en `http://127.0.0.1:5000`

9. Extra 2 — Aplicación visual con Streamlit

DE MODELO A PRODUCTO

La interfaz convierte probabilidades técnicas en decisiones accionables para recepción y Revenue Management.

9.1 Funcionalidades

Zona	Función
Predicción individual	Formulario por hotel, calendario, huéspedes, servicios, garantía y canal; devuelve probabilidad, riesgo y recomendación.
Predicción por lotes	Carga CSV o muestra de 30 reservas, calcula KPIs, distribución de riesgo, detalle y exportación enriquecida.
Benchmark técnico	Lee metrics_summary.csv, muestra métricas, curva ROC, importancia y matriz de confusión.
Sidebar	Identidad del equipo, ganador dinámico, métricas, acceso a MLflow y umbrales operativos.

9.2 Decisiones de ingeniería

- `st.cache_resource` carga una sola vez el Random Forest de aproximadamente 414 MB y evita penalizar cada interacción.
- `st.cache_data` reutiliza el resumen de métricas y detecta dinámicamente el ganador.
- La misma función `predict_reservation` acepta dict para tiempo real o DataFrame para lotes, reduciendo duplicación.
- `OneHotEncoder(handle_unknown="ignore")` mejora la robustez ante categorías no vistas por el modelo.

9.3 Flujo de uso

1. Activar el entorno virtual y ejecutar `streamlit run app.py`.
2. Abrir `http://localhost:8501` y completar una reserva o cargar un CSV.
3. Revisar la probabilidad, el nivel de riesgo y la acción sugerida.
4. Auditar el rendimiento en la pestaña de métricas y abrir MLflow cuando sea necesario.

10. Conclusiones vinculadas a las clases

10.1 Aprendizajes técnicos

Los resultados reproducen varias ideas centrales del módulo. Un baseline sencillo es imprescindible: la regresión logística ofrece una referencia clara, pero su menor AUC evidencia que la cancelación combina umbrales e interacciones. El árbol aprende esas reglas, aunque un único estimador es inestable. Random Forest, mediante bagging, reduce esa varianza y obtiene el mejor equilibrio. XGBoost, mediante boosting, maximiza recall y queda muy cerca del ganador.

La red neuronal confirma que mayor complejidad no garantiza mejor rendimiento. Dropout y EarlyStopping controlan sobreajuste, pero en datos tabulares con 119 mil ejemplos los ensambles de árboles aprovechan mejor las relaciones heterogéneas y requieren menos decisiones de arquitectura. La conclusión coincide con el enfoque de clase: se comparan modelos bajo el mismo split y las métricas deciden, no la sofisticación percibida.

10.2 Aprendizajes sobre evaluación

Accuracy no puede interpretarse sola: un clasificador que siempre predijera “no cancela” ya alcanzaría aproximadamente 62,96 %. ROC-AUC evalúa ordenación en todos los umbrales y F1 obliga a equilibrar precision y recall. La matriz de confusión traduce las métricas a casos concretos y facilita discutir el coste de 931 falsas alarmas frente a 1.569 cancelaciones perdidas.

10.3 Conclusión de negocio

El modelo no sustituye la decisión del hotel: prioriza reservas para actuar. Su mayor valor aparece al combinar probabilidad, coste y ocupación. Con un umbral más bajo se detectan más cancelaciones; con uno más alto se reduce el riesgo de sobreventa. El sistema deja esta decisión abierta y ofrece tres niveles operativos comprensibles.

CONCLUSIÓN FINAL

La práctica cumple el ciclo completo visto en clase: entender datos, prevenir leakage, transformar variables, entrenar modelos comparables, validar con métricas adecuadas, seleccionar con evidencia y llevar el resultado hasta una interfaz utilizable.

11. Reflexión crítica, limitaciones y mejoras

11.1 Limitaciones actuales

- El split es aleatorio. Una validación temporal entrenando con años anteriores y probando en periodos posteriores estimaría mejor el cambio de demanda.
- Las probabilidades no se han calibrado explícitamente. Un AUC alto no garantiza que 70 % signifique siete cancelaciones de cada diez.
- Los umbrales 40 % y 70 % son reglas de negocio iniciales; deben optimizarse con costes reales de habitación vacía, compensación y reputación.
- La importancia por impureza puede favorecer variables continuas o de alta cardinalidad y no representa causalidad.
- Agrupar países y agentes controla dimensionalidad, pero puede ocultar segmentos pequeños con riesgo particular.
- El modelo ocupa alrededor de 414 MB. Conviene medir latencia, memoria y alternativas más ligeras antes de desplegar a escala.
- El histórico cubre 2015–2017 y la interfaz simula reservas futuras; existe riesgo de drift de precios, canales y comportamiento.
- La red neuronal puede mostrar pequeñas variaciones entre ejecuciones pese a fijar semillas, por paralelismo y operaciones numéricas.

11.2 Mejoras priorizadas

1. Validación temporal y backtesting por temporada y tipo de hotel.
2. Calibración Platt o isotónica, curva de calibración y Brier score.
3. Optimización del umbral mediante una matriz de costes acordada con Revenue Management.
4. Permutation importance y SHAP para dirección, estabilidad e interpretación local.
5. Monitorización de drift de entrada, distribución de scores y métricas reales en MLflow.
6. Compresión o destilación del modelo y medición de rendimiento de la aplicación.
7. Pruebas automatizadas de esquema, preprocesado e inferencia y, si se requiere integración, API REST.

12. Reproducibilidad y entrega

12.1 Ejecución

```
ENTRENAMIENTO
cd practica-machine-learning
source .venv/bin/activate
python -m src.train --tune
```

```
SERVICIOS
mlflow ui          # http://127.0.0.1:5000
streamlit run app.py # http://localhost:8501
```

12.2 Artefactos generados

Artefacto	Uso
reports/metrics_summary.csv	Comparación ordenada de métricas.
reports/figures/roc_curve_comparison.png	ROC conjunta de los cinco modelos.
reports/figures/confusion_matrix_*.png	Errores y aciertos del ganador.
reports/figures/roc_curve_*.png	Curva ROC del ganador.
reports/figures/feature_importance_*.png	Variables más utilizadas por el bosque.
models/preprocessor.joblib	Transformaciones ajustadas con train.
models/best_model.joblib	Modelo seleccionado automáticamente.
mlflow.db / mlruns	Historial y artefactos de experimentación.

12.3 Repositorio

<https://github.com/TFM-JUPITER-ASISTENTE-INDUSTRIAL/practica-machine-learning>

Anexo A — Trazabilidad con la rúbrica

Criterio	Evidencia en la práctica
Implementación y comparación (30 %)	Cinco modelos obligatorios, mismo split, tuning y tabla común.
Evaluación y visualización (25 %)	ROC-AUC/F1 justificados, matriz, ROC conjunta e importancia.
Automatización y estructura (25 %)	Pipeline modular, selección automática, persistencia e inferencia.
Código y documentación (20 %)	README, requirements, docstrings, carpetas y comandos reproducibles.
Bonus	MLflow Tracking e interfaz Streamlit funcionales.

Fuentes internas consultadas

- Guión “PontIA - Master IA - Machine Learning - Práctica Evaluación Final - 2026-02”.
- README.md y módulos src/ del proyecto.
- notebooks/01_eda.ipynb.
- Jupyter de Clases: modelos supervisados y clasificación binaria.
- Jupyter de Clases: Keras/TensorFlow, Dropout y EarlyStopping.
- reports/metrics_summary.csv, figuras del pipeline y registros de MLflow.

Documento generado a partir de los artefactos verificables de la ejecución actual.